YOTSUBA SHADER

Syntax reference

Every concept of the language in the HLSL, GLSL, MSL, SPIR-V and WGSL syntaxes

Edition 1.0 · October 2026 · Generated from the attribute sources of the YotsubaShader.<X>Syntax projects

Yotsuba Shader lets you write the same shader with the names of five shading languages. Each set of names is a **syntax**, shipped as its own package with a namespace of the same name. They all map to the same concepts, so the compiler produces the same intermediate representation whichever one you use. The syntax never decides the output: the compilation targets are chosen separately, in the `<YotsubaTargets>` property of your project.

The five syntax packages

Syntax	Package and namespace	Resources namespace	Inspired by	Attributes
HLSL	`YotsubaShader.HlslSyntax`	`YotsubaShader.HlslSyntax.Resources`	HLSL (Direct3D)	24
GLSL	`YotsubaShader.GlslSyntax`	`YotsubaShader.GlslSyntax.Resources`	GLSL (OpenGL / Vulkan GLSL)	34
MSL	`YotsubaShader.MslSyntax`	`YotsubaShader.MslSyntax.Resources`	Metal Shading Language	30
SPIR-V	`YotsubaShader.SpirvSyntax`	`YotsubaShader.SpirvSyntax.Resources`	SPIR-V (Vulkan)	33
WGSL	`YotsubaShader.WgslSyntax`	`YotsubaShader.WgslSyntax.Resources`	WGSL (WebGPU)	23

Neutral core: YotsubaShader.Language

These attributes are the same in every syntax. Engines and frameworks build on this core; each syntax package only adds names.

Concept	Attribute
Interface location (vertex inputs, varyings, fragment outputs)	`[Location(n)]`
Uniform buffer (neutral spelling)	`[Uniform]`
Storage buffer (neutral spelling)	`[StorageBuffer]`
Set and binding (neutral spelling)	`[Binding(set, binding)]`
Shader stage of a method	`[ShaderStage(Stage.Vertex | Fragment | Compute)]`

Built-in values

Concept	HLSL	GLSL	MSL	SPIR-V	WGSL
Vertex stage					
Clip-space position (vertex output)	`SV_Position`	`gl_Position`	`position`	`BuiltIn_Position`	`@builtin_position`
Vertex index	`SV_VertexID`	`gl_VertexIndex`	`vertex_id`	`BuiltIn_VertexIndex`	`@builtin_vertex_index`
Instance index	`SV_InstanceID`	`gl_InstanceIndex`	`instance_id`	`BuiltIn_InstanceIndex`	`@builtin_instance_index`
Base vertex	`SV_StartVertexLocation`	`gl_BaseVertex`	`base_vertex`	`BuiltIn_BaseVertex`	—

Concept	HLSL	GLSL	MSL	SPIR-V	WGSL
Base instance	`SV_StartInstanceLocation`	`gl_BaseInstance`	`base_instance`	`BuiltIn_BaseInstance`	—
Clip distance	`SV_ClipDistanceN`	`gl_ClipDistance`	`clip_distance`	`BuiltIn_ClipDistance`	`@builtin_clip_distances`
Render target layer	`SV_RenderTargetArrayIndex`	`gl_Layer`	`render_target_array_index`	`BuiltIn_Layer`	—
Viewport index	`SV_ViewportArrayIndex`	`gl_ViewportIndex`	`viewport_array_index`	`BuiltIn_ViewportIndex`	—
Fragment stage					
Fragment coordinate	`SV_Position`	`gl_FragCoord`	`position`	`BuiltIn_FragCoord`	`@builtin_position`
Front-facing flag	`SV_IsFrontFace`	`gl_FrontFacing`	`front_facing`	`BuiltIn_FrontFacing`	`@builtin_front_facing`
Fragment depth (output)	`SV_Depth`	`gl_FragDepth`	`depth`	`BuiltIn_FragDepth`	`@builtin_frag_depth`
Conservative depth	`SV_DepthGreaterEqual` `SV_DepthLessEqual`	—	`depth("greater")` `depth("less")`	—	—
Stencil reference (output)	`SV_StencilRef`	`gl_FragStencilRefARB`	`stencil`	`BuiltIn_FragStencilRef`	—
Sample index	`SV_SampleIndex`	`gl_SampleID`	`sample_id`	`BuiltIn_SampleId`	`@builtin_sample_index`
Sample mask	`SV_Coverage`	`gl_SampleMask` `gl_SampleMaskIn`	`sample_mask`	`BuiltIn_SampleMask`	`@builtin_sample_mask`
Primitive index	`SV_PrimitiveID`	`gl_PrimitiveID`	`primitive_id`	`BuiltIn_PrimitiveId`	`@builtin_primitive_index`
Barycentric coordinates	`SV_Barycentrics`	`gl_BaryCoordEXT`	`barycentric_coord`	`BuiltIn_BaryCoord`	—
Barycentric coordinates (no perspective)	—	`gl_BaryCoordNoPerspEXT`	—	`BuiltIn_BaryCoordNoPersp`	—
Helper invocation	—	`gl_HelperInvocation`	—	`BuiltIn_HelperInvocation`	—
Sample position	—	`gl_SamplePosition`	—	`BuiltIn_SamplePosition`	—
Compute stage					
Global invocation id	`SV_DispatchThreadID`	`gl_GlobalInvocationID`	`thread_position_in_grid`	`BuiltIn_GlobalInvocationId`	`@builtin_global_invocation_id`
Local invocation id	`SV_GroupThreadID`	`gl_LocalInvocationID`	`thread_position_in_threadgroup`	`BuiltIn_LocalInvocationId`	`@builtin_local_invocation_id`
Local invocation index	`SV_GroupIndex`	`gl_LocalInvocationIndex`	`thread_index_in_threadgroup`	`BuiltIn_LocalInvocationIndex`	`@builtin_local_invocation_index`
Workgroup id	`SV_GroupID`	`gl_WorkGroupID`	`threadgroup_position_in_grid`	`BuiltIn_WorkgroupId`	`@builtin_workgroup_id`
Number of workgroups	—	`gl_NumWorkGroups`	`threadgroups_per_grid`	`BuiltIn_NumWorkgroups`	`@builtin_num_workgroups`
Workgroup size (method attribute)	`numthreads`	`layout_local_size`	—	—	`@workgroup_size`
Workgroup size (built-in value)	—	`gl_WorkGroupSize`	`threads_per_threadgroup`	`BuiltIn_WorkgroupSize`	—
Subgroup size	—	`gl_SubgroupSize`	`threads_per_simdgroup`	`BuiltIn_SubgroupSize`	`@builtin_subgroup_size`
Subgroup invocation id	—	`gl_SubgroupInvocationID`	`thread_index_in_simdgroup`	`BuiltIn_SubgroupInvocationId`	`@builtin_subgroup_invocation_id`
Subgroup id	—	`gl_SubgroupID`	`simdgroup_index_in_threadgroup`	`BuiltIn_SubgroupId`	`@builtin_subgroup_id`
Number of subgroups	—	`gl_NumSubgroups`	`simdgroups_per_threadgroup`	`BuiltIn_NumSubgroups`	`@builtin_num_subgroups`

Resources and binding slots

Concept	HLSL	GLSL	MSL	SPIR-V	WGSL
Uniform buffer	`cbuffer, ConstantBuffer<T>`	`uniform`	`constant`	`Uniform`	`uniform`
Read/write storage buffer	`RWStructuredBuffer<T>`	`buffer`	`device`	`StorageBuffer`	`storage(storage_use.read_write)`
Read-only storage buffer	`StructuredBuffer<T>`	`buffer`	`device(readOnly: true)`	`StorageBuffer`	`storage(storage_use.read)`

Concept	HLSL	GLSL	MSL	SPIR-V	WGSL
Binding slot	`register(RegisterPrefix.b, n)`	`layout(set, binding)`	`buffer(n)`, `texture(n)`, `sampler(n)`	`DescriptorSet(s)` + `Binding(n)`	`@group(s)` + `@binding(n)`
Texture slot	`register(RegisterPrefix.t, n)`	`layout(set, binding)`	`texture(n)`	`DescriptorSet(s)` + `Binding(n)`	`@group(s)` + `@binding(n)`
Sampler slot	`register(RegisterPrefix.s, n)`	`layout(set, binding)`	`sampler(n)`	`DescriptorSet(s)` + `Binding(n)`	`@group(s)` + `@binding(n)`

Notes

- **Position and fragment coordinate.** `SV_Position`, `position` and `@builtin_position` are one attribute for both: on a vertex output it is the clip-space position, on a fragment input it is the fragment coordinate. GLSL and SPIR-V have a separate name for each (`gl_Position` / `gl_FragCoord`, `BuiltIn_Position` / `BuiltIn_FragCoord`).
- **Workgroup size.** HLSL, GLSL and WGSL declare it with a method attribute (`numthreads`, `layout_local_size`, `@workgroup_size`). MSL and SPIR-V expose it as a built-in value that the shader reads.
- **Several spellings in one cell** (for example `gl_SampleMask` / `gl_SampleMaskIn`) are the output and input forms of the same concept, or equivalent variants.
- **One syntax per file.** Some names exist in more than one syntax (`uniform`, `buffer`). Import one syntax namespace per file to keep them unambiguous.
- **Which backend supports each concept** is in the companion PDF, *Backend compatibility*.